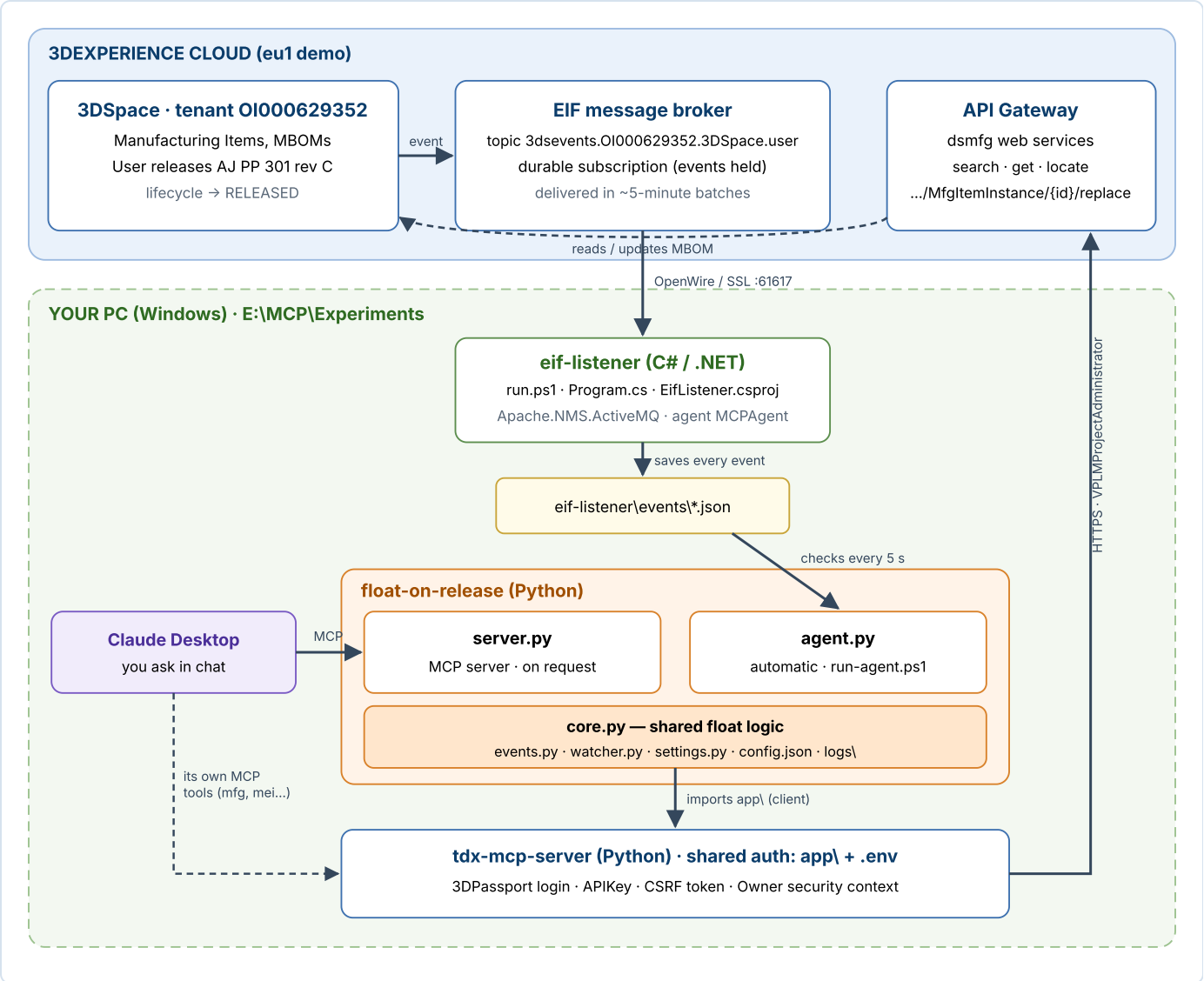
Float on Release

Architecture of the automatic MBOM revision-float agent for 3DEXPERIENCE tenant OI000629352 · October 14, 2026

Purpose

When a new revision of a Manufacturing Item is **released**, Float on Release finds every immediate parent MBOM that still uses an **older revision** of that item and re-points those instances to the released revision. It runs automatically on the 3DEXPERIENCE EIF *Status Change* event. It can also be run on request from Claude Desktop as an MCP server.

Architecture diagram



Solid arrows: data flow for an automatic float. Dashed arrows: other API traffic.

Components

1. eif-listener (C#, .NET 7+)

- **Files:** `Program.cs` (the listener), `EifListener.csproj` (build file: .NET version, the `Apache.NMS.ActiveMQ` broker client, copy `config.json`), `run.ps1` (start script; asks for the MCPAgent API key unless it is set in `EIF_AGENT_SECRET` or `agent.secret`).
- **What it does:** connects to the EIF broker as the EIF agent with a **durable** subscription and saves every message as `events\yyyyMMdd-HHmmss-fff_NNNN.json`. It does no filtering, so new event types need no code change.
- **Why C#:** the broker speaks OpenWire, and the supported client library is .NET only. That is why the system is in two parts: the C# listener receives, and Python acts.

2. Float on Release (Python)

File	Role
`agent.py`	**The automatic agent.** Started by `run-agent.ps1`. Watches the events folder and floats every release. Single-job options: `--item "AJ PP 301"` (preview), `--item … --apply`, `--event file.json`.
`server.py`	**The MCP server** for Claude Desktop. Tools: `preview_float_on_release`, `float_on_release(apply)`, `handle_eif_event`, `recent_float_runs`, `float_on_release_settings`. `--http --watch` runs the server and the agent in one process.
`float_on_release\core.py`	The float logic shared by both: find older revisions → where-used → check the parent's state → replace.
`events.py`	Decides whether an event is a Manufacturing Item `statusChanged` to `RELEASED`.
`watcher.py`	Checks the events folder and remembers which files it has handled.
`settings.py` / `config.json`	Settings (see below).
`logs\`	`events_seen.log` (every event and whether it matched), `runs.jsonl` (every run and its result), `processed_events.json`.

3. tdx-mcp-server (Python)

- Your general 3DEXPERIENCE MCP server in Claude Desktop (mfg, mei, qual, doc, ug, rt tools).
- Float on Release **imports its `app\` client** for 3DPassport login, API key, CSRF token and API Gateway routing. All credentials stay in that project's single `.env` file.

4. 3DEXPERIENCE services used

Call (via API Gateway, `/api/dsmfg/v1`)	Used for
`GET dsmfg:MfgItem/{id}` (Details mask)	Live state of the item; parent state and reservation
`GET dsmfg:MfgItem/search?$searchStr=<name>`	All revisions of the item (same name and type)
`POST dsmfg:MfgItemInstance/locate` (filterResults=target)	Where-used: instances of the older revisions and their parents
`POST dsmfg:MfgItem/{parent}/dsmfg:MfgItemInstance/{inst}/replace`	Re-point the instance to the released revision

How an automatic float works

1. A user releases a Manufacturing Item revision in 3DEXPERIENCE (for example AJ PP 301 rev C).
2. 3DSpace publishes an EIF event: `eventClass` "Manufacturing Item", `eventType` "statusChanged", `predicate` "state", `object.Value` "RELEASED", and `subject.identifier` = the item's physical ID. It arrives in the next ~5-minute batch.
3. The eif-listener saves the event to `events\*.json`.
4. `agent.py` sees the new file within 5 seconds. `events.py` keeps only Manufacturing Item status changes to Released; everything else is logged and skipped.
5. `core.py` confirms the item's **live** state is RELEASED, then finds its **older** revisions (same name and type, created earlier). Newer revisions are never downgraded.
6. Instance locate returns each instance of an older revision and its **immediate parent**. Parents in a state not allowed by the settings (by default, Obsolete), or reserved by another user, are skipped and reported.
7. For each remaining instance, `replace` is called with the Owner security context. 3DEXPERIENCE deletes the old instance and creates a new one with the same name pointing to the released revision.
8. The result is written to `logs\runs.jsonl`.

Configuration (`float-on-release\config.json`)

Setting	Current value	Meaning
`security_context`	VPLMProjectAdministrator.Company Name.Common Space	Owner role. Allows updating Released parents in place, without a new parent revision.
`dry_run`	false	The automatic agent applies changes. `true` = log only. Read at start-up; Claude Desktop ignores it and uses the tool's `apply` flag.
`editable_parent_states`	PRIVATE, IN_WORK, FROZEN, RELEASED	Parent states that may be changed; Obsolete parents are skipped.
`status_event_types`	[] (auto-detect)	Optional exact event types; the real value is `statusChanged`.
`events_dir`	../eif-listener/events	Where the listener saves events.
`process_existing_events`	false	On the very first start, events already in the folder are treated as history.
`change_action_id`	(empty)	Optional Change Action, for parents under change control.

What runs, and how

Process	Started by	Needed for
EIF listener	`eif-listener\run.ps1`	Automatic floats (must stay running)
Float on Release agent	`float-on-release\run-agent.ps1`	Automatic floats (must stay running)
Float on Release MCP server	Claude Desktop (`claude_desktop_config.json`)	Floats or previews on request
tdx-mcp-server	Claude Desktop	General 3DEXPERIENCE tools

Run **either** `run-agent.ps1` **or** `server.py --http --watch`, never both, otherwise a release could be handled twice. If the listener or agent is stopped, nothing is lost: the broker holds events for the durable subscription, and the agent catches up on unprocessed files.

Operational notes

- **Latency:** about 5 minutes from release to float, almost all of it waiting for the EIF batch. The agent itself acts within seconds.
- **Re-runs are safe:** once parents point to the released revision, there is nothing left to replace.
- **Instance IDs change** after a replace (the name stays, e.g. "AJ PP 301.0"). Search-based views (search, Data Perspective) lag the BOM by a few minutes; check the revision column, not the name.
- **Not yet set up:** start-up with Windows. Both windows must currently stay open; a Task Scheduler setup would also need the agent key stored for unattended start.

Verification, October 14, 2026

Time (ET)	Test	Result
15:55	AJ PP 301 rev B released (automatic agent, dry run)	Event matched; planned AJ MA 300: rev A → B; nothing changed
15:59	`agent.py --item "AJ PP 301" --apply`	AJ MA 300 (Released) instance now points to rev B
16:10	Preview from Claude Desktop	MCP tools working; 0 left to replace
16:15	AJ PP 301 rev C released (automatic agent, live)	AJ MA 300 updated from rev B to rev C automatically, with no manual step

Folders: E:\MCP\Experiments\eif-listener · E:\MCP\Experiments\float-on-release · E:\MCP\Experiments\tdx-mcp-server